Current Progress Overview

This report focuses on the current progress of data preprocessing and model experimentation.

The team is currently developing two main approaches:

Image-based model: detecting eye state from eye images.

EAR time-series model: detecting blink / eye-closure behavior from EAR sequences.

Current Processing Flow:

Video / Frame → Eye Image → EAR Sequence →

Image Model → EAR Model → Result Comparison

Image Preprocessing: Two Separate Pipelines

◎ Shared Input Split

- **Input after splitting:**
  train_split.csv
  val_split.csv
  test_split.csv

- These split files are created once and reused by both image preprocessing pipelines.

▣ Two Image Preprocessing Pipelines

Pipeline	Used for	Main input	Output
SVM preprocessing	Traditional ML	HOG / HOG + EAR	Feature vectors
CNN preprocessing	Deep Learning	Eye image 24×24	Image tensors

◎ Key Idea / Takeaway

The same data split is used, but each pipeline converts the eye image into a different representation.

SVM Preprocessing: HOG + EAR

Split CSV → Grayscale → Resize 24×24 → Normalize /255 → HOG → Add EAR → StandardScaler → SMOTE → Save .npy

Input & Pipeline

Input

- train_split.csv
- test_split.csv

Pipeline

- Read eye image as grayscale
- Resize image to 24×24
- Normalize pixel values by /255
- Extract HOG features
- Append ear_avg
- Apply StandardScaler
- Apply SMOTE on training set only

Feature & Output

Feature

- feature = HOG features + ear_avg

Output files

- X_train_img.npy
- y_train_img.npy
- X_test_img.npy
- y_test_img.npy
- scaler_img.pkl

Purpose **This pipeline converts each eye image into a numerical feature vector suitable for SVM and other traditional Machine Learning models.**

SVM Preprocessing: HOG-only

Split CSV → Grayscale → Resize 24×24 → Normalize /255 → HOG only → StandardScaler → SMOTE → Save .npy

Purpose & Pipeline

1. **Purpose**
   - Evaluate the real contribution of EAR.
   - This version removes EAR and keeps only HOG features.
   - The result will be compared with the HOG + EAR version.

2. **Pipeline**
   - Read eye image as grayscale
   - Resize image to 24×24
   - Normalize pixel values by /255
   - Extract HOG features only
   - Apply StandardScaler
   - Apply SMOTE on training set only
   - Save processed .npy files

Feature & Comparison

1. **Feature**

   feature = HOG

2. **Comparison**

Version	Feature
SVM HOG-only	HOG
SVM HOG+EAR	HOG + EAR

3. **Key idea**

The HOG-only pipeline is used as an ablation experiment to check whether adding EAR really improves model performance.

CNN Preprocessing

Input & Pipeline

1. **Input**
- train_split.csv
- val_split.csv
- test_split.csv

2. **Pipeline**
- Read eye image as grayscale
- Resize image to 24×24
- Normalize pixel values to [0, 1]
- Expand image shape to (24, 24, 1)
- Save processed .npy files

Output & Key Difference

1. **Output files**

File	Description
X_train_cnn.npy	Training image tensors
y_train_cnn.npy	Training labels
X_val_cnn.npy	Validation image tensors
y_val_cnn.npy	Validation labels
X_test_cnn.npy	Test image tensors
y_test_cnn.npy	Test labels

2. **Key difference**

- CNN does not require HOG or EAR features.
- CNN learns visual features directly from the eye image.
- The input is an image tensor, not a handcrafted feature vector.

IMAGE MODEL

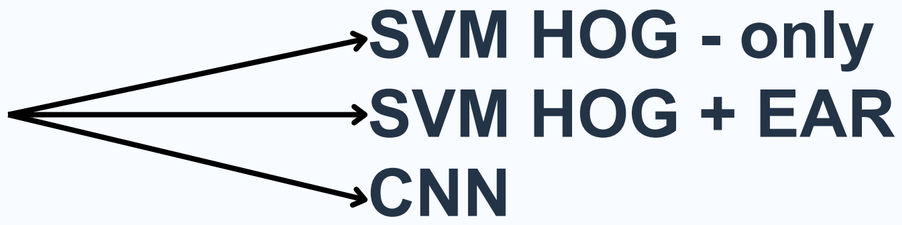


Image Model: Training SVM Models

Two SVM experiments:

Model	Input Feature	Purpose
SVM HOG-only	HOG	Baseline / ablation study
SVM HOG+EAR	HOG + EAR	Check if EAR improves performance

Data used

Set	Ratio
Train	~64%
Test	~20%

Training setup

Model: SVC
Kernel: RBF
C = 10
gamma = scale
probability = True

Class imbalance:
SMOTE is applied only on the training set.

Output models

svm_hog_only_model.pkl
svm_model.pkl

Image Model: Training CNN Model

Data used

Set	Ratio
Train	~64%
Validation	~16%
Test	~20%

Why CNN uses validation set?

Validation set is used for:
- EarlyStopping
- ModelCheckpoint
- Threshold tuning

CNN architecture

Input 24×24×1
→ Data Augmentation
→ Conv2D + BatchNorm + MaxPooling
→ Conv2D + BatchNorm + MaxPooling
→ Dense + Dropout
→ Sigmoid output

Training setup:

Optimizer: Adam
Learning rate: 0.0003
Loss: Binary Crossentropy
Monitor: val_auc

Class imbalance strategy

No SMOTE
No final class weight
Use validation-based threshold tuning

Image Model: Experimental Results

Model	Accuracy	AUC	Open F1	Closed F1	Macro F1
SVM HOG-only	0.8888	0.9471	0.9294	0.7384	0.8339
SVM HOG+EAR	0.8911	0.9483	0.9309	0.7423	0.8366
CNN threshold=0.45	0.9086	0.9313	0.9424	0.7781	0.8603

Latency

Model	Latency
SVM HOG+EAR	1.1162 ms/image
SVM HOG-only	**1.0181 ms/image**
CNN	9.8176 ms/image

Confusion Matrix

Model	True Open → Open	True Closed → Closed
SVM HOG-only	1297	278
SVM HOG+EAR	1301	278
CNN threshold=0.45	1326	284

Threshold Tuning & Future Improvements

CNN Threshold Tuning

CNN output: prob_closed
Default threshold: 0.50
Best threshold: 0.45
Tuned on validation set

Why tune threshold?
- Dataset is imbalanced
- Closed class is smaller
- 0.50 is not always the best
- Validation set is used, not test set

Future Improvements

- Retrain final SVM with train + validation
- Try HOG + normalized EAR
- Use calibration for EAR
- Add sequence logic for blink detection
- Use temporal smoothing for realtime stability

EAR Sequence Preprocessing Pipeline

Objective: Prepare continuous EAR sequences for blink / eye-closure detection over time. EAR is calculated from eye landmarks in each frame.

Pipeline Steps

1. Noise Filtering & Linear Interpolation
2. Hybrid Dynamic Calibration
3. Video-based Split
4. Sliding Window
5. Ground-Truth Label



EAR Sequence Preprocessing

1. Noise Filtering

- **Remove invalid frames:** Camera failed to track face, faulty frames, duplicate frames.
- After removing invalid frames, the EAR sequence may become discontinuous.
- **Linear interpolation** is used to fill these gaps and keep the EAR sequence continuous.

EAR Sequence Preprocessing

2. Hybrid Dynamic Calibration

Biometric Diversity:
Each person has different eye characteristics (large eyes, narrow eyes, monolids).

No Global Min-Max:
Min-Max Scaler requires the entire video's data, causing unrealistic "future leakage".
→ Capture the maximum EAR in the first 1 second (15 frames) to establish the initial 100% openness baseline

Dynamic Updating:
If a wider eye opening is detected later, the system dynamically updates this new peak as the new 100% baseline.

Zero-Division Prevention:
If eyes remain perfectly static (Max = Min), Min-Max Scaler crashes due to division by zero.
→ Using “Current / Max”.

Cold-Start Protection:
What if the eyes are closed at startup?
→ If the initial EAR is extremely low (< 0.18), set a safe default maximum (0.23)

EAR Sequence Preprocessing

3. Video-based Split

- **Data Leakage Problem:** Randomly splitting sliding windows causes the AI to accidentally "peek" at Test data during Training, resulting in inflated, fake accuracy.

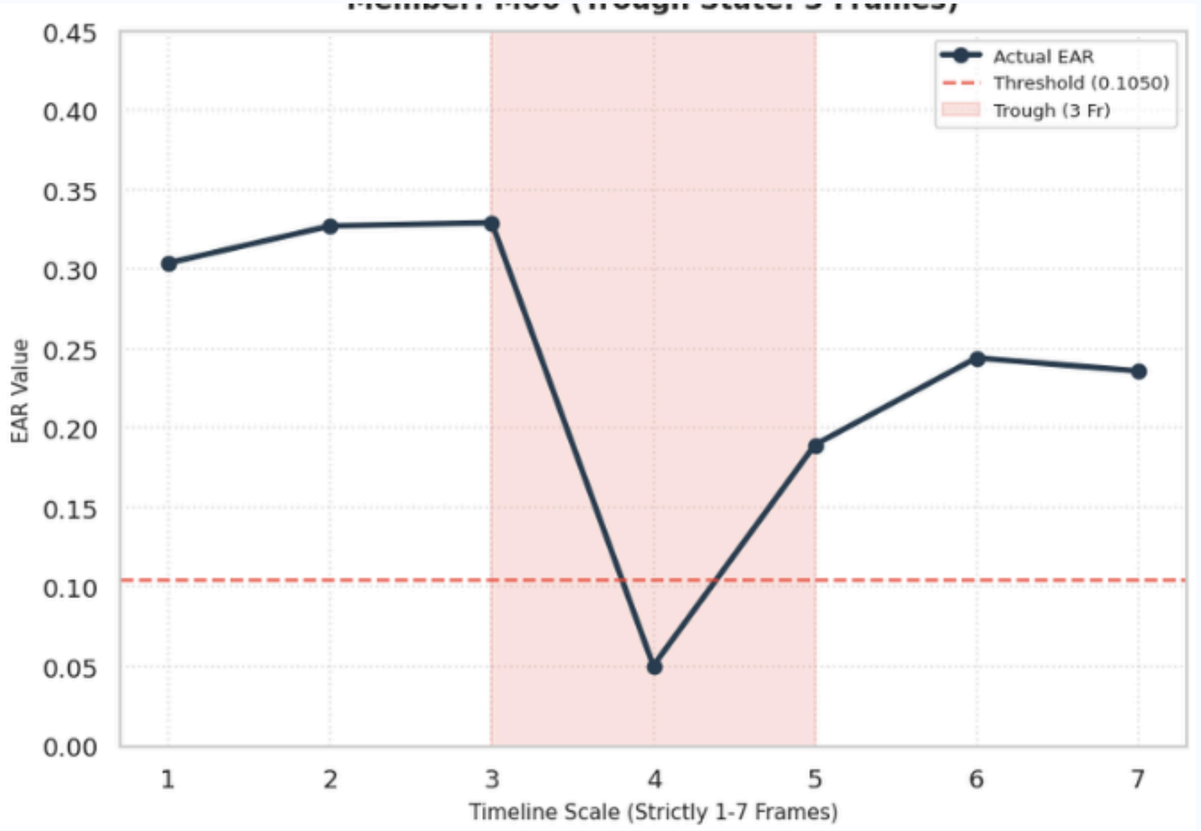
- We split the dataset by Video IDs. 20 entire videos are reserved for training, and 5 completely different videos are strictly isolated for testing.
- 80/20 train/test split based on the video ~ 78.6/21.4 train/test train/test split based on frame sequences.
-



EAR Sequence Preprocessing

4. Sliding Window

- Instead of using isolated frames, data is divided into short segments of 7 consecutive frames (~0.46 seconds), step size = 1.
- This allows the model to learn the movement pattern of the eye during blinking or eye closure over time.



EAR Sequence Preprocessing

5. Ground-truth Labeling

Initial idea:

- Using threshold: EAR < 0.1050

⇒ This missed many blink cases from people with naturally narrow eyes.

Final Solution:

- Refactored to use ground-truth labels from metadata.
- Helped recover additional blink and long-closure samples missed by the static threshold method.

Sequence Labeling Logic:

- **Label 0 (No-Blink)**: Normal open eyes or transitioning states.
- **Label 1 (Blink)**: 1 to 4 consecutive closed frames (Successfully captures ultra-fast, single-frame blinks).
- **Label 2 (Long-Closure)**: > 4 consecutive closed frames (Clear indicator of severe drowsiness).

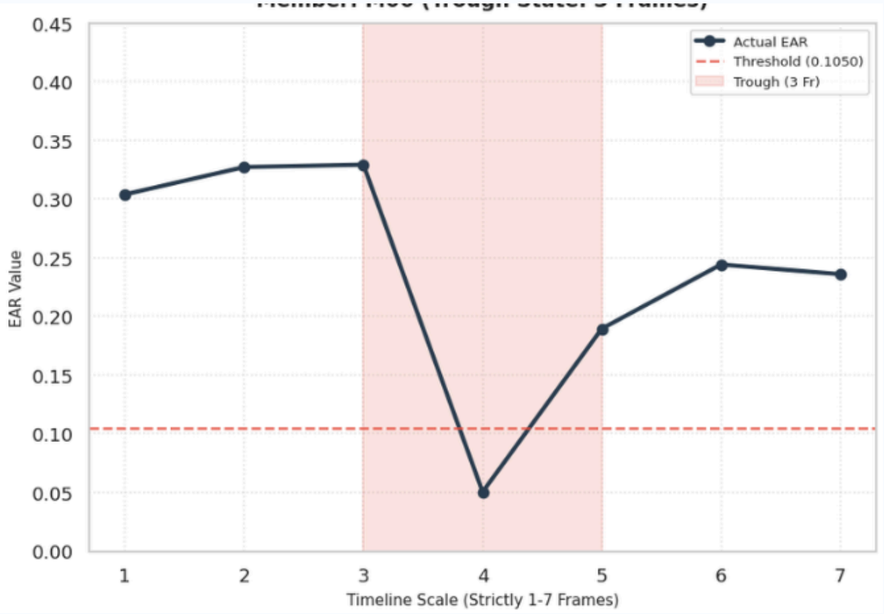
EAR Sequence Preprocessing

LABEL COMPARISON

Label	Static EAR	Ground-truth	Difference
0 - no blink	3355	3133	-222
1 - blink	287	442	+155
2 - long closure	354	421	+67

Time Series EAR Approach

Goal: Classify 3 states (No-Blink, Blink, Long-Closure) using 7-frame sliding window.



1. Baseline: raw_7frame_ear

- Input: 7 raw EAR values directly.
- X_raw = [0.95,0.65,0.35,0.05,0.25,0.70,0.90]

2. Improved: advanced_12feature

- Input: Raw EAR + Min + Max + Std + Ratio_center + Kurtosis (12 dimensions).
- X_adv = [0.95,0.65,0.35,0.05,0.25,0.70,0.90, **0.05**,**0.95**,**0.315**,**18.5**,**−1.357**]

Time Series EAR Approach

Why Advanced Features Overperform:

- **Raw EAR is Non-linear:** Blink shape is a V-curve. Raw features require non-linear models (RBF SVM) which are slow (30.5 ms).
- **Feature Linearization:** Calculating 5 dynamic features simplifies the decision space.
- **Linear Model Benefit:** Simple models (Linear SVM, Logistic Regression) run extremely fast (7.0 ms) and achieve higher F1 (0.9049).

Model	Feature	Hyperparameter	Acc	Macro F1	Time
Linear SVM	**advanced_12feature**	**C=1.0**	**0.9334**	**0.9049**	**0.0070s**
RBF SVM	raw_7frame_ear	C=0.5	0.9308	0.8937	0.0310s
Linear SVM	advanced_12feature	C=5.0	0.9283	0.8923	0.0070s
Linear SVM	advanced_12feature	C=0.1	0.9296	0.8908	0.0090s
Logistic Regression	advanced_12feature	balanced	0.927	0.8896	0.0010s
RBF SVM	raw_7frame_ear	C=1.5	0.9283	0.8837	0.0280s
RBF SVM	advanced_12feature	C=0.5	0.927	0.8821	0.0327s
RBF SVM	advanced_12feature	C=1.5	0.9193	0.8797	0.0290s
RBF SVM	raw_7frame_ear	C=5.0	0.9116	0.871	0.0268s
Linear SVM	raw_7frame_ear	C=5.0	0.9129	0.8607	0.0090s

Time Series EAR Approach

3. Hybrid Model

Approach: Use a heuristic filter to reduce computation.

min (EAR window) > t

If true, predict label 0 (No Blink). Only when false, execute the ML model.

Threshold t	% ML Bypassed	Accuracy	Macro F1
0.5	**54.50%**	**0.9309**	**0.8952**
0.4	57.40%	0.9296	0.8859
0.35	62.40%	0.9065	0.8527
0.3	69.40%	0.886	0.8132
0.2	90.40%	0.7746	0.5342

Effectiveness: Skips ML inference for 54.5% of real-world windows. Reduces CPU workload drastically and supports real-time deployment.

THANH YOU FOR LISTENING